

[CPE223] COMPUTER ARCHITECTURES

M1 Mock Exam

Date: 17 February 2026 | Time: 09:00 - 12:00

Part 1: Multiple Choice Questions (20 Points)

Instructions: Select the correct answer.

1. Which statement is **true**?
 - a) The motherboard contains the circuitry that connects the processor to the other hardware
 - b) The term *bit* refers to the amount of memory required to store one letter
 - c) When you press a keyboard key, the character is stored on the computer's hard disk until it is needed by the program.
 - d) If a computer has a 64-bit data bus, it can transfer 4 bytes of data at a time.
2. Which character is **not** Condition Codes ?
 - a) N (negative)
 - b) V (overflow)
 - c) E (equal)
 - d) Z (zero)
3. Which addressing mode does **not** access the main memory ?
 - a) Auto-increment
 - b) Relative
 - c) Direct
 - d) Register
4. The C-language statement, $A = *B+1$, can be compiled into (OPERATION SOURCE DES)
 - a) Move 1(B), A
 - b) Move (B), A; Add #1, A
 - c) Move B, A; Add #1, A
 - d) Add 1, B; Move B, A

5. Which statement is **false** ?

- a) $5 = (0000101)_{1'compliment}, (0000101)_{2'compliment}$
- b) $-19 = (1101100)_{1'compliment}, (1101101)_{2'compliment}$
- c) $-43 = (1010101)_{1'compliment}, (1010110)_{2'compliment}$
- d) $-10 = (1110101)_{1'compliment}, (1110110)_{2'compliment}$

6. Which of the following addition generates overflow ?

- a) $11011 + 00111$
- b) $10110 + 10011$
- c) $11101 + 11000$
- d) $00101 + 01010$

7. Which statement is **true** ?

- a) Using a stack to save the return address of a subroutine supports subroutine nesting but not sufficient for recursion.
- b) `ORIGIN 2000`
`DATAWORD 1000`
The above statements cause 1000 to be placed at location 2000 during the program execution
- c) It is possible to tell whether a binary pattern represents a machine instruction or an operand.
- d) None of the above

8. Which statement regarding Byte Ordering (Endianness) is **TRUE**?

- a) Big-Endian stores the least significant byte (LSB) at the lowest memory address.
- b) Little-Endian stores the least significant byte (LSB) at the lowest memory address.
- c) The value 0x1234 would be stored as 34 12 in Big-Endian.
- d) Network protocols (TCP/IP) typically standardise on Little-Endian.

9. What is the process of removing layers from the surface of a silicon wafer during chip manufacturing called?

- a) Etching
- b) Crafting
- c) Baking
- d) Doping

10. Which register holds the address of the **next** instruction to be fetched?
- a) IR
 - b) PC
 - c) MAR
 - d) MDR
11. Where is the instruction that is **currently being executed** stored?
- a) PC
 - b) MAR
 - c) IR
 - d) ALU
12. Writing data to a **Special Purpose Register** (e.g., Program Counter) incorrectly is likely to cause:
- a) The computer to explode.
 - b) An automatic shutdown.
 - c) The hard disk to format itself.
 - d) An unexpected instruction execution or branching error.
13. Which addressing mode is most suitable for implementing **Pointers** in high-level languages?
- a) Immediate Addressing
 - b) Direct Addressing
 - c) Indirect Addressing
 - d) Relative Addressing
14. In the instruction Load R1, 16(R2), if R2 = 1000, what is the **Effective Address**?
- a) 1000
 - b) 1016
 - c) 16
 - d) 2000
15. Which statement regarding Program Translation is **TRUE**?
- a) A Compiler translates assembly language into machine code.
 - b) An Assembler translates assembly language into machine code.
 - c) A Linker combines source code files into a text file.
 - d) An Interpreter executes the high-level code directly without any translation ever happening.

16. What does the directive ORIGIN 200 mean?
- a) The program execution starts at line 200.
 - b) It defines a constant variable named ORIGIN with value 200.
 - c) The next block of code/data will be placed at memory address 200.
 - d) It reserves 200 bytes of memory.
17. In a polling I/O mechanism, what is the function of the status flag (e.g., SIN/SOUT)?
- a) To interrupt the processor immediately.
 - b) To indicate that the I/O device is ready for data transfer.
 - c) To clear the input buffer.
 - d) To signal that the power is on.
18. Which symbol represents a **Non-blocking assignment** in Verilog?
- a) =
 - b) <=
 - c) ==
 - d) :=
19. Which statement about Verilog HDL is **TRUE**?
- a) Blocking assignments (=) are executed concurrently (in parallel).
 - b) Non-blocking assignments (<=) are typically used for combinational logic.
 - c) A flip-flop is a sequential circuit element that changes state on a clock edge.
 - d) The "initial" block runs continuously throughout the simulation.
20. What is the meaning of always @(posedge clk)?
- a) Execute continuously when clock is 1.
 - b) Execute once when clock changes from 0 to 1.
 - c) Execute whenever clock changes state.
 - d) Execute when clock is 0.

Part 2: Short-answer questions (20 Points)

Instructions: Briefly answer or explain the following concepts. (Lecture 1-3 only, No Verilog)

1. What does **ALU** stand for?
2. What does **MDR** stand for?
3. Briefly explain the function of the **Program Counter (PC)**.
4. Briefly explain the function of the **Instruction Register (IR)**.
5. What does **MAR** stand for?
6. What does **RISC** stand for?
7. Explain the concept of **Direct Addressing Mode**.
8. What is the **Stack Pointer (SP)** used for?
9. Given the value `0x12345678`, if stored in memory at address 1000 using **Little Endian** format, write down the byte value at address 1000, 1001, 1002, and 1003.
10. Consider the following assembly loop:

```
MOV    R1, #5
LOOP:  SUB    R1, #1
      CMP    R1, #0
      BGE    LOOP
```

Explain the operation and state the **condition** that causes the loop to **terminate**.

Part 3: Problem Solving (20 Points)

1. (10 Points) The table below shows the current content of the memory and the stack. Write the **NEW** content in both memory and stack after execution.

The content of the memory

1000	1002
1002	1004
1004	1006
1006	1008
1008	1010
1010	1012

The content of the stack

SP	1000
	1002
	1008

After the execution of the assembly program below, write the **new content** in both memory and stack.

```
Load    R0, #1004
Load    R1, #1006
Load    R2, (SP)
Add     SP, #1
Load    R3, (SP)
Add     SP, #1
Subtract SP, #1
Store   (SP), (R0)+2
Add     R2, ((R1))
Add     R2, (R3)+4
Store   R1, R2
```

Note: DO NOT just give the answer. Please explain how you compute the answer so that partial points can be awarded even if the final answer is wrong.

2. (10 Points) Translate the following C program into Assembly Language.

Given C Code (Filter Positive Numbers):

```
int N = 9; // Index for source array 'src' int J = 0; // Index for destination array 'dst' // Source Array (mix of positive
and negative) int src[10] = {5, -20, 35, -4, 50, -6, 72, -8, 9, -100};
int dst[10]; // Destination Array
while (N >= 0) {
    // If element is positive, copy it to dst
    if (src[N] > 0) {
        dst[J] = src[N];
        J++;
    }
    N--;
}
```

Instructions:

- Use register **R1** for **N** and **R2** for **J**.
- Use **R3** for base address of **src** and **R4** for base address of **dst**.
- You must calculate the correct byte offset for both arrays using the indices.
- Use conditional branches.

Answer Space:

Answer Key (เฉลยละเอียด)

Part 1: Multiple Choice Answers

ข้อที่	เฉลย	คำอธิบาย (ภาษาไทย)
1	A	Motherboard เชื่อมต่อ Hardware ส่วนต่าง ๆ เข้าด้วยกัน
2	C	E (Equal) เป็น condition suffix ไม่ใช่ Flag (Flags คือ N, Z, V, C)
3	D	Register Mode อ่านข้อมูลจาก Register ใน CPU ไม่ต้องออกไป Main Memory
4	B	Move (B), A (ย้ายค่าที่ B ซึ่มา A) แล้ว Add #1, A (บวก 1)
5	B	ข้อ B ผิด เพราะ 1's complement ของ -19 คือ 11101100 (ไม่ใช่ 1101100)
6	B	$10110(-10) + 10011(-13)$ ได้ -23 ซึ่งเกิน range 5-bit (-16) เกิด Overflow
7	B	ORIGIN 2000 กำหนดให้ข้อมูลถัดไป (1000) เริ่มที่ Address 2000
8	B	Little-Endian เก็บ Byte นัยสำคัญต่ำสุด (LSB) ไว้ที่ Address ต่ำสุด
9	A	Etching (การกัดกรวด) เป็นขั้นตอนเอา Layer ออกจาก Wafer
10	B	PC (Program Counter) เก็บ Address ของคำสั่งถัดไป
11	C	IR (Instruction Register) เก็บคำสั่งที่กำลังถูก execute ปัจจุบัน
12	D	การเขียนทับ Special Register อาจทำให้การทำงานผิดพลาด (Execution error)
13	C	Indirect Addressing ใช้ค่าใน Reg/Mem เป็น Pointer ชี้ไปยังข้อมูลจริง
14	B	Effective Address = $1000(R2) + 16 = 1016$
15	B	Assembler แปลง Assembly เป็น Machine Code (Compiler แปลง High-level)
16	C	ORIGIN 200 บอกว่า Block ถัดไปให้เริ่มที่ Address 200
17	B	Status Flag ใช้บอกว่าอุปกรณ์ I/O พร้อมรับ/ส่งข้อมูลแล้ว (Ready)
18	B	$\leq$ เป็นสัญลักษณ์ของ Non-blocking assignment ใน Verilog
19	C	Flip-flop เป็นวงจร Sequential เปลี่ยนสถานะตามขอบขา Clock
20	B	posedge clk คือช่วงที่สัญญาณนาฬิกาเปลี่ยนจาก 0 ขึ้นไป 1

Part 2: Short Answer Solutions

1. **ALU:** หน่วยคำนวณทางคณิตศาสตร์และตรรกะ (Arithmetic and Logic Unit)
2. **MDR:** Memory Data Register เก็บข้อมูลที่เพิ่งอ่านมาจากหรือกำลังจะเขียนลง Memory
3. **PC:** เก็บ Address ของคำสั่งถัดไปที่จะถูก Fetch มาทำงาน
4. **IR:** เก็บ Instruction ปัจจุบันที่กำลังถูก Decode/Execute อยู่
5. **MAR:** Memory Address Register เก็บ Address ที่ต้องการติดต่อกับ Memory
6. **RISC:** สถาปัตยกรรมคอมพิวเตอร์ที่เน้นชุดคำสั่งขนาดเล็กและเรียบง่าย (Reduced Instruction Set Computer)
7. **Direct Addressing:** การระบุ Memory Address ลงไปตรง ๆ ในตัวคำสั่งเลย (เช่น LOAD R1, 1000)

8. **SP:** ชี้ตำแหน่งบนสุดของ Stack (Top of Stack) ใช้เก็บข้อมูลชั่วคราวหรือ Return Address
9. **Little Endian:** ที่ Address 1000 เก็บ Byte นัยสำคัญต่ำสุด (LSB)
 - Address 1000: **78**
 - Address 1001: **56**
 - Address 1002: **34**
 - Address 1003: **12**
10. **Code Explanation:** ลูปนี้จะลดค่า R1 ทีละ 1 และวนซ้ำตราบเท่าที่ $R1 \geq 0$ (วนลูปตามจำนวนรอบที่กำหนดใน R1 หรือจนกว่า R1 จะติดลบ)

Part 3: Problem Solving Solutions

1. Memory & Stack Content Step-by-Step Trace

สมมติฐาน: การบวก/ลบ SP ด้วย #1 หมายถึงการเลื่อนไป Address ถัดไปตามขนาด Word (2 bytes) ในระบบจำลองนี้

1. **Initial State:** SP = 1000, Mem[1000]=1002, Mem[1002]=1004, ...
2. Load R0, #1004: R0 = 1004
3. Load R1, #1006: R1 = 1006
4. Load R2, (SP): อ่าน Mem[1000] $\rightarrow$ R2 = 1002
5. Add SP, #1: SP เลื่อนลง $\rightarrow$ SP = 1002 (Address ถัดไป)
6. Load R3, (SP): อ่าน Mem[1002] $\rightarrow$ R3 = 1004
7. Add SP, #1: SP เลื่อนลง $\rightarrow$ SP = 1004
8. Subtract SP, #1: SP ย้อนกลับ $\rightarrow$ SP = 1002
9. Store (SP), (R0)+2:
 - ปลายทาง: (SP) คือ Mem[1002]
 - ต้นทาง: ใช้ R0 (1004) เป็น Address $\rightarrow$ อ่าน Mem[1004] ได้ค่า **1006**
 - หลังอ่าน R0 บวกเพิ่ม 2 $\rightarrow$ R0 = 1006 (Post-increment)
 - **ผลลัพธ์:** เขียนค่า 1006 ทับลงใน Mem[1002] (ค่าเดิมคือ 1004 หายไป)
10. Add R2, ((R1)):
 - R1 = 1006
 - Value = Mem[Mem[1006]] = Mem[1008] = 1010
 - R2 = R2 + 1010 = 1002 + 1010 = **2012**
11. Add R2, (R3)+4:
 - R3 ปัจจุบันคือ 1004 (โหลดไว้ตั้งแต่ข้อ 6 ก่อน Mem[1002] จะเปลี่ยน และ R3 ไม่ได้ถูกแก้)
 - อ่าน Mem[R3] = Mem[1004] = 1006
 - R2 = R2 + 1006 = 2012 + 1006 = **3018**
 - R3 บวกเพิ่ม 4 $\rightarrow$ R3 = 1008

12. Store R1, R2:

- เขียนค่า R2 (3018) ลงใน Address ที่ R1 ชี้ (1006)
- ผลลัพธ์: Mem[1006] เปลี่ยนเป็น 3018

Final Answers:

Memory	
1000	1002
1002	1006
1004	1006
1006	3018
1008	1010
1010	1012

Stack
(SP อยู่ที่ 1002)
1000
1002
1008

2. C to Assembly Translation (Filter Positive Numbers)

Logic : ใช้ R1(N), R2(J), R3(Base src), R4(Base dst). ต้องคำนวณ Offset แยกกัน 2 ตัว.

SRC_DATA	DCD	5, -20, 35, -4, 50, -6, 72, -8, 9, -100	; Source Array
DST_DATA	SPACE	40	; Allocate 40 bytes (10 ints)
START:	LDR	R1, =9	; N = 9 (Index for src)
	MOV	R2, #0	; J = 0 (Index for dst)
	LDR	R3, =SRC_DATA	; Base Address src
	LDR	R4, =DST_DATA	; Base Address dst
LOOP:	CMP	R1, #0	; Loop Condition: N >= 0
	BLT	DONE	; If N < 0, Exit Loop
	LSL	R5, R1, #2	; Calculate Offset for src: R5 = N * 4
	LDR	R6, [R3, R5]	; Load src[N] into R6
	CMP	R6, #0	; Check if src[N] > 0
	BLE	SKIP_COPY	; If negative or zero, skip copy
	LSL	R7, R2, #2	; Calculate Offset for dst: R7 = J * 4
	STR	R6, [R4, R7]	; Store src[N] into dst[J]
	ADD	R2, R2, #1	; Increment J (J++)
SKIP_COPY:	SUB	R1, R1, #1	; Decrement N (N--)
	B	LOOP	; Repeat Loop
DONE:			; Program End